

Betriebssysteme

- Übung 3: ZFS -

Franz Pawlus

1422169

22. März 2026

1 Betriebssysteme - Übung 3: Aufgabe 1

1.1 Einleitung

In der ersten Aufgabe des dritten Übungsblatts sollte ein einfaches Sicherungs- und Archivierungsprogramm mit Hilfe von ZFS implementiert werden. Die zentrale Anforderung bestand dabei einen Ordner **X** auf eine möglichst konsistente Weise zu sichern. Dabei sollten explizit ZFS-Snapshots sowie die Funktionen **zfs send** und **zfs receive** verwendet werden.

Ich habe mich dazu entschieden, die Lösung in Python zu implementieren und den zu sichernden Ordner **X** nicht als beliebiges Verzeichnis auf einem klassischen Dateisystem zu behandeln, sondern als Mountpoint eines dedizierten ZFS-Datasets. Für meine Lösung habe ich eine Ubuntu VM unter VirtualBox genutzt. In dieser VM habe ich **zfsutils-linux** installiert und mehrere virtuelle Disks angelegt, um damit eigene ZFS-Pools und Datasets aufzubauen.

Bei meiner Implementierung wird pro Programmdurchlauf ein Snapshot des Quell-Datasets erstellt, anschließend per **zfs send** in eine **.zfs**-Datei exportiert und in einem konfigurierbaren Zielverzeichnis **Y** gespeichert. Für die Wiederherstellung habe ich einen separaten Restore-Pfad mit **zfs receive** implementiert. Zusätzlich wird eine einfache, aber kontrollierte Retention-Strategie umgesetzt, die nur die neuesten **N** Backup-Dateien beibehält.



Abbildung 1: Schematische Darstellung meiner Implementierung.

Wichtig hervorzuheben ist, dass die Wiederherstellung nicht direkt in das Quell-Dataset erfolgt, sondern in ein separates Restore-Dataset. Ich habe mich dafür entschieden, um die Live-Daten nicht versehentlich zu überschreiben und um den gesicherten Zustand sauber mit dem aktuellen Zustand vergleichen zu können.

1.2 Nutzung des Programms

Ein typischer Backup-Aufruf sieht so aus:

```
1 sudo /home/franz/.venvs/zfs-backup-tool/bin/python -m
   zfs_backup_tool.cli backup --config /media/sf_bung03/config.
   local.json
```

Dabei wird:

- die Konfiguration geladen,
- ein Snapshot des Quell-Datasets erzeugt,
- und der Snapshot als **.zfs**-Datei in das konfigurierte Zielverzeichnis exportiert.

Ein typischer Restore-Aufruf sieht so aus:

```
1 sudo /home/franz/.venvs/zfs-backup-tool/bin/python -m
   zfs_backup_tool.cli restore --config /media/sf_bung03/config.
   local.json --backup-file /home/franz/zfs_backups/backup_2026
   -03-21_17-26-40.zfs
```

Hierbei wird die angegebene `.zfs`-Datei mit `zfs receive` in das konfigurierte Restore-Dataset eingespielt.

Beide Aufrufe arbeiten mit einer JSON-Konfigurationsdatei. Diese enthält die aus meiner Sicht minimal nötigen Parameter:

```
1 {
2   "source_dataset": "sourcepool/data",
3   "source_mountpoint": "/sourcepool/data",
4   "backup_directory": "/home/franz/zfs_backups",
5   "max_backups": 5,
6   "snapshot_prefix": "backup",
7   "restore_dataset": "backuppool/restore"
8 }
```

2 Betriebssysteme - Übung 3: Aufgabe 2

In der zweiten Aufgabe sollte ich durch ein konkretes Experiment zeigen, dass ZFS-Snapshots zwar einen technischen konsistenten Zustand eines Dateisystems sichern, dass dies aber nicht automatisch bedeutet, dass auch die Inhalte aus Sicht einer laufenden Anwendung logisch vollständig sind. Genau dieser Unterschied zwischen Dateisystemkonsistenz und Anwendungskonsistenz war für mich der eigentliche Kern der Aufgabe.

Ich habe mich für das Experiment bewusst gegen eine Datenbank und für ein kleines eigenes Python-Skript entschieden. Mir war wichtig, den Effekt möglichst kontrolliert und nachvollziehbar zu erzeugen, ohne zusätzliche Komplexität einzuführen, die vom eigentlichen Problem ablenkt. Das Skript hält eine Datei während eines laufenden Schreibvorgangs offen und schreibt einen logischen Datensatz absichtlich in zwei Phasen. Dadurch ließ sich sehr kontrolliert ein Zeitpunkt erzeugen, in dem der Snapshot zwar korrekt ist, die Datei aus Sicht der Anwendung aber noch nicht vollständig abgeschlossen wurde.

2.1 Versuchsaufbau

Für das Experiment habe ich die Infrastruktur aus Aufgabe 1 wiederverwendet. Das heißt:

- das Quelldataset ist weiterhin `sourcepool/data`
- die Sicherung wird weiterhin als `.zfs`-Datei in `/home/franz/zfs\_backups` gespeichert
- die Wiederherstellung erfolgt weiterhin in das Restore-Dataset `backuppool/restore`

Zusätzlich habe ich dafür ein eigenes Experiment-Skript implementiert:

- `experiments/aufgabe2\_inconsistent\_writer.py`

Dieses Skript schreibt in die Datei:

```
1 /sourcepool/data/live_transaction.log
```

Der logische Datensatz wird dabei nicht in einem Schritt geschrieben, sondern in zwei Phasen:

1. Zunächst wird nur der erste Teil des Datensatzes geschrieben, zum Beispiel:

```
1 tx=1 payload=100 checksum=ad5736686512
```

2. Danach wird der Puffer bewusst mit `flush()` und `fsync()` auf das Dateisystem geschrieben.
3. Anschließend wartet das Skript für einige Sekunden und öffnet damit ein sichtbares Inkonsistenzfenster.
4. Erst danach wird der Datensatz mit

```
1 COMMIT
```

vervollständigt.

Wichtig ist dabei, dass ein Datensatz in meinem Experiment nur dann als logisch vollständig gilt, wenn er mit `COMMIT` endet.

2.2 Durchführung des Experiments

Das Experiment wurde in zwei Terminalfenstern innerhalb der Ubuntu-VM durchgeführt.

Im ersten Terminal habe ich den Writer gestartet. Dieser schrieb zunächst den ersten Teil des Datensatzes und meldete anschließend sichtbar, dass das Inkonsistenzfenster geöffnet ist. Besonders wichtig war mir dabei, dass der erste Teil des Datensatzes bereits wirklich auf dem Dateisystem liegt. Deshalb wird er vor der Pause bewusst mit `flush()` und `fsync()` geschrieben. Während genau dieses Fensters habe ich im zweiten Terminal mit dem bestehenden Backup-Programm aus Aufgabe 1 einen Snapshot erzeugt und als `.zfs`-Datei gespeichert.

Danach durfte der Writer weiterlaufen und den Datensatz in der Live-Datei vollständig beenden. Anschließend habe ich die erzeugte `.zfs`-Datei mit `zfs receive` in das Restore-Dataset zurückgespielt und die Live-Datei mit der wiederhergestellten Datei verglichen.

Das Ergebnis war genau der gewünschte Nachweis. Die Live-Datei enthielt nach Abschluss des Schreibvorgangs:

```
1 tx=1 payload=100 checksum=ad5736686512 COMMIT
```

Die aus der Sicherung wiederhergestellte Datei enthielt dagegen nur:

```
1 tx=1 payload=100 checksum=ad5736686512
```

Der Snapshot und auch das daraus erzeugte Backup waren also technisch korrekt. Die Datei existierte, sie war lesbar, und das Backup ließ sich problemlos wiederherstellen. Trotzdem war der letzte logische Datensatz in der Sicherung unvollständig, weil der Snapshot genau zwischen Phase 1 und Phase 2 des Schreibvorgangs erstellt wurde.

Damit zeigt das Experiment, dass ZFS-Snapshots zwar Crash-Konsistenz liefern, aber nicht automatisch Anwendungskonsistenz. Das Dateisystem befindet sich in einem gültigen Zustand, die Anwendung hatte ihren eigenen logischen Schreibvorgang zu diesem Zeitpunkt aber noch nicht abgeschlossen.

2.3 Möglichkeiten zur Verbesserung

Um diesen Nachteil zu reduzieren, gibt es mehrere sinnvolle Ansätze.

Eine Möglichkeit ist, die betreffende Anwendung vor dem Snapshot kurz zu pausieren oder in einen ruhenden Zustand zu versetzen. Dadurch wird verhindert, dass sich gerade ein halbfertiger Datensatz im Schreibvorgang befindet.

Eine weitere Möglichkeit ist, die Anwendung vor dem Snapshot gezielt zu einem Flush oder Checkpoint zu veranlassen. Gerade bei Datenbanken ist das aus meiner Sicht der professionellere Weg, weil dort oft eigene Mechanismen existieren, um Transaktionen sauber abzuschließen oder zumindest in einen wiederherstellbaren Zustand zu bringen.

Darüber hinaus können Anwendungen selbst robuster schreiben, etwa indem sie zunächst in eine temporäre Datei schreiben und diese erst nach vollständigem Abschluss atomar umbenennen. Dadurch sinkt das Risiko, dass ein Snapshot gerade eine logisch unfertige Datei einfriert.

3 Betriebssysteme - Übung 3: Aufgabe 3

In der dritten Aufgabe sollte ich zeigen, dass der Ausfall einer Platte in einem abgesicherten ZFS-Pool keinen unmittelbaren Einfluss auf das Dateisystem hat und dass Anwendungen, die in diesem Moment bereits lesend und schreibend auf Dateien zugreifen, ohne Unterbrechung weiterarbeiten können.

Ich habe mich dafür bewusst für einen eigenen separaten Testpool entschieden, um die bereits funktionierende Umgebung aus Aufgabe 1 nicht zu gefährden. Außerdem habe ich `raidz` statt `raidz2` gewählt, weil für die geforderte Demonstration der Toleranz gegenüber einem einzelnen Plattenausfall bereits ein einfacher RAIDZ-Verbund ausreicht und sich das Experiment dadurch klarer und kompakter durchführen ließ.

Für das Experiment habe ich in der VirtualBox-VM vier zusätzliche 5-GB-Platten zu einem eigenen RAIDZ-Pool zusammengefasst:

- /dev/sdd
- /dev/sde
- /dev/sdf
- /dev/sdg

3.1 Durchführung des Experiments

Das Experiment lief in zwei Terminalfenstern innerhalb der Ubuntu-VM.

Im ersten Terminal lief das Workload-Skript und erzeugte fortlaufend neue erfolgreiche Iterationen. Im zweiten Terminal habe ich parallel mit

```
1 watch -n 1 zpool status raidpool
```

den Zustand des Pools beobachtet.

Während das Skript bereits aktiv auf /raidpool/data/workload.log arbeitete, habe ich auf Host-Seite in VirtualBox die zu /dev/sdg gehörende hotplug-fähige Platte an SATA-Port 7 aus der laufenden VM entfernt. Damit wurde ein echter virtueller Plattenausfall innerhalb des RAIDZ-Verbunds simuliert.

3.2 Beobachtung und Ergebnis

Das Ergebnis war eindeutig.

Nach dem Entfernen der Platte war /dev/sdg im Gast nicht mehr vorhanden. Gleichzeitig meldete ZFS für den Pool:

```
1 pool: raidpool
2 state: DEGRADED
```

und im Detail:

```
1 sdg      REMOVED
```

Trotzdem lief das Workload-Skript weiter. Auch nach dem Ausfall wurden weiterhin erfolgreiche Iterationen erzeugt. Zusätzlich blieb die Datei /raidpool/data/workload.log lesbar, und die letzten Zeilen zeigten, dass nach dem Ausfall weitergeschrieben wurde.

Das Verhalten des Pools ist genau das, was man von **raidz** erwarten würde. Der Verbund verliert durch den Ausfall einer Platte zwar einen Teil seiner Redundanz und wird deshalb nur noch im Zustand **DEGRADED** betrieben, bleibt aber weiterhin funktionsfähig.

4 Betriebssysteme - Übung 3: Aufgabe 4

In der vierten Aufgabe sollte die Zugriffsperformanz von ZFS mit dem Standard-Linux-Dateisystem ext4 verglichen werden. Für den Vergleich habe ich zwei separate Benchmark-Ziele eingerichtet und diese anschließend mit `fio` getestet.

Ich habe mich bewusst auf drei Szenarien beschränkt: 1. Sequentielles Schreiben, 2. Sequentielles Lesen und 3. Gemischter 4K-Zufallszugriff

Die gemessenen Werte waren in meinem Setup relativ eindeutig.

Test	Dateisystem	Lauf 1	Lauf 2	Lauf 3
Sequentielles Schreiben	ZFS	260 MiB/s	282 MiB/s	269 MiB/s
Sequentielles Schreiben	ext4	999 MiB/s	2040 MiB/s	2103 MiB/s
Sequentielles Lesen	ZFS	464 MiB/s	481 MiB/s	469 MiB/s
Sequentielles Lesen	ext4	1036 MiB/s	1082 MiB/s	1208 MiB/s
4K-Random Read	ZFS	1192 IOPS	1281 IOPS	1317 IOPS
4K-Random Read	ext4	1311 IOPS	1295 IOPS	1294 IOPS
4K-Random Write	ZFS	1186 IOPS	1275 IOPS	1310 IOPS
4K-Random Write	ext4	1304 IOPS	1288 IOPS	1287 IOPS

Tabelle 1: Vergleich der gemessenen Benchmark-Ergebnisse von ZFS und ext4.

Beim sequentiellen Schreiben und Lesen war ext4 in meinem Setup deutlich schneller als ZFS. Beim gemischten 4K-Zufallszugriff lagen beide Dateisysteme deutlich näher beieinander, ext4 blieb aber auch dort leicht vorne. Auffällig war außerdem, dass ext4 beim sequentiellen Schreiben stärker schwankte als ZFS.

Aus meiner Sicht passt dieses Ergebnis grundsätzlich zur Architektur der beiden Dateisysteme. ext4 ist im Vergleich zu ZFS einfacher aufgebaut und muss bei Schreibzugriffen weniger Zusatzarbeit für Mechanismen wie Copy-on-Write, zusätzliche Metadatenaktualisierung und Integritätsmechanismen leisten. ZFS bietet dafür andere Stärken, etwa Snapshots, Prüfbarkeit und Flexibilität im Umgang mit Pools und Datasets, erkaufte sich diese Eigenschaften aber in meinem Versuch durch messbaren Overhead.

Wichtig ist mir dabei, das Ergebnis nicht zu überdehnen. Mein Experiment zeigt nicht, dass ext4 generell immer besser ist. Es zeigt nur, dass ext4 in genau diesem virtuellen Testaufbau und mit genau diesen drei `fio`-Workloads die höheren Messwerte geliefert hat.